

COMPLETE PYTHON + LOGIC + DSA FOUNDATION

60 Problems with Code + Interview Explanation + Complexity

1. Reverse String

```
s = "shivam"  
print(s[::-1])
```

Concept: Reverse sequence using slicing.

Pattern: String Manipulation

Time Complexity: $O(n)$

2. Palindrome String

```
s = "madam"  
print(s == s[::-1])
```

Concept: Check if string equals its reverse.

Pattern: Two Pointer / String

Time Complexity: $O(n)$

3. Factorial

```
n = 5  
fact = 1  
for i in range(1, n + 1):
```

```
fact *= i
print(fact)
```

Concept: Multiplication sequence.

Pattern: Loop / Recursion

Time Complexity: $O(n)$

4. Fibonacci Series

```
a, b = 0, 1
for _ in range(10):
    print(a, end=" ")
    a, b = b, a + b
```

Concept: Each number depends on previous values.

Pattern: Dynamic Sequence

Time Complexity: $O(n)$

5. Prime Number

```
n = 13
is_prime = True
for i in range(2, int(n**0.5)+1):
    if n % i == 0:
        is_prime = False
print(is_prime)
```

Concept: Check divisibility till square root.

Pattern: Math Optimization

Time Complexity: $O(\sqrt{n})$

6. Count Vowels

```
s = "interview"  
print(sum(1 for c in s if c in "aeiou"))
```

Concept: Traverse string and validate condition.

Pattern: String Traversal

Time Complexity: $O(n)$

7. Find Maximum

```
arr = [4,7,1,9]  
print(max(arr))
```

Concept: Find largest element.

Pattern: Array Traversal

Time Complexity: $O(n)$

8. Find Minimum

```
arr = [4,7,1,9]  
print(min(arr))
```

Concept: Find smallest element.

Pattern: Array Traversal

Time Complexity: $O(n)$

9. Second Largest

```
arr = [1,5,3,9,7]
arr = list(set(arr))
arr.sort()
print(arr[-2])
```

Concept: Remove duplicates then sort.

Pattern: Sorting / Traversal

Time Complexity: $O(n \log n)$

10. Remove Duplicates

```
arr = [1,2,2,3,4]
print(list(set(arr)))
```

Concept: Use set for uniqueness.

Pattern: HashSet

Time Complexity: $O(n)$

11. Character Frequency

```
s = "banana"
freq = {}
for c in s:
    freq[c] = freq.get(c,0)+1
print(freq)
```

Concept: Track frequency counts.

Pattern: HashMap

Time Complexity: $O(n)$

12. Sort List

```
arr = [5,2,8,1]
print(sorted(arr))
```

Concept: Sort using built-in Timsort.

Pattern: Sorting

Time Complexity: $O(n \log n)$

13. Bubble Sort

```
arr = [5,1,4,2]
for i in range(len(arr)):
    for j in range(len(arr)-1-i):
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
print(arr)
```

Concept: Swap adjacent elements repeatedly.

Pattern: Nested Loop Sorting

Time Complexity: $O(n^2)$

14. Linear Search

```
arr = [2,4,6,8]
target = 6
print(target in arr)
```

Concept: Sequential searching.

Pattern: Traversal

Time Complexity: $O(n)$

15. Binary Search

```
arr = [1,2,3,4,5,6]
target = 5
l, r = 0, len(arr)-1
while l <= r:
    mid = (l+r)//2
    if arr[mid] == target:
        print("Found")
        break
    elif arr[mid] < target:
        l = mid+1
    else:
        r = mid-1
```

Concept: Search sorted array efficiently.

Pattern: Divide & Conquer

Time Complexity: $O(\log n)$

16. Anagram

```
s1 = "listen"
s2 = "silent"
print(sorted(s1) == sorted(s2))
```

Concept: Compare sorted strings.

Pattern: String Comparison

Time Complexity: $O(n \log n)$

17. Swap Numbers

```
a, b = 5, 10
a, b = b, a
print(a, b)
```

Concept: Tuple unpacking.

Pattern: Variable Manipulation

Time Complexity: $O(1)$

18. Word Count Dictionary

```
text = "python is easy and python is powerful"
words = text.split()
count = {}
for w in words:
    count[w] = count.get(w,0)+1
print(count)
```

Concept: Track word frequencies.

Pattern: HashMap

Time Complexity: $O(n)$

19. Largest Word

```
text = "I love python programming"
print(max(text.split(), key=len))
```

Concept: Find longest string.

Pattern: Max with Key

Time Complexity: $O(n)$

20. Common Elements

```
a = [1,2,3]
b = [2,3,4]
print(list(set(a)&set(b)))
```

Concept: Use set intersection.

Pattern: HashSet

Time Complexity: $O(n)$

21. Merge Dictionaries

```
d1 = {"a":1}
d2 = {"b":2}
print(**d1, **d2)
```

Concept: Combine mappings.

Pattern: Dictionary Operations

Time Complexity: $O(n)$

22. String Compression

```
s = "aaabbc"
res = ""
count = 1
for i in range(1, len(s)):
    if s[i] == s[i-1]:
        count += 1
    else:
        res += s[i-1] + str(count)
        count = 1
res += s[-1] + str(count)
print(res)
```


Concept: Run-length encoding.

Pattern: String Compression

Time Complexity: $O(n)$

23. Move Zeros

```
arr = [0,1,0,3,12]
res = [x for x in arr if x != 0]
res.extend([0]*arr.count(0))
print(res)
```

Concept: Maintain order while moving zeros.

Pattern: Array Rearrangement

Time Complexity: $O(n)$

24. Rotate List

```
arr = [1,2,3,4,5]
k = 2
print(arr[-k:] + arr[:-k])
```

Concept: Circular shifting.

Pattern: Array Manipulation

Time Complexity: $O(n)$

25. Sum of Digits

```
n = 1234
print(sum(int(d) for d in str(n)))
```

Concept: Digit extraction.

Pattern: Math / Traversal

Time Complexity: $O(n)$

26. Armstrong Number

```
n = 153
power = len(str(n))
print(sum(int(d)**power for d in str(n)) == n)
```

Concept: Power sum validation.

Pattern: Math Logic

Time Complexity: $O(n)$

27. Two Sum

```
arr = [2,7,11,15]
target = 9
seen = {}
for i, num in enumerate(arr):
    diff = target - num
    if diff in seen:
        print([seen[diff], i])
    seen[num] = i
```

Concept: HashMap lookup.

Pattern: HashMap

Time Complexity: $O(n)$

28. Valid Parentheses

```
s = "()[]{}"
stack = []
```

```
mp = {'(': '(', ')': ')', '[': '[', ']' : ']', '{': '{', '}': '}' }
valid = True
for c in s:
    if c in "([{"):
        stack.append(c)
    else:
        if not stack or stack.pop() != mp[c]:
            valid = False
print(valid)
```

Concept: Stack-based matching.

Pattern: Stack

Time Complexity: $O(n)$

29. List Comprehension

```
arr = [1,2,3,4]
print([x*x for x in arr if x%2==0])
```

Concept: Compact transformation syntax.

Pattern: Pythonic Iteration

Time Complexity: $O(n)$

30. Matrix Transpose

```
matrix = [[1,2,3],[4,5,6]]
transpose = [[row[i] for row in matrix] for i in range(len(matrix[0]))]
print(transpose)
```

Concept: Swap rows and columns.

Pattern: Matrix Manipulation

Time Complexity: $O(\text{rows} \times \text{cols})$

31. Reverse Integer

```
n = 1234
print(int(str(n)[::-1]))
```

32. Even/Odd

```
n = 7
print("Even" if n%2==0 else "Odd")
```

33. Find Duplicates

```
arr = [1,2,3,2,4]
seen = set()
dup = set()
for i in arr:
    if i in seen:
        dup.add(i)
    seen.add(i)
print(list(dup))
```

34. Missing Number

```
arr = [1,2,4,5]
n = 5
print(n*(n+1)//2 - sum(arr))
```

35. Merge Lists

```
a = [1,2]
b = [3,4]
print(a+b)
```

36. Flatten Nested List

```
arr = [[1,2],[3,4],[5]]
print([x for sub in arr for x in sub])
```

37. Unique Element

```
arr = [1,1,2,2,3]
res = 0
for i in arr:
    res ^= i
print(res)
```

38. Count Digits

```
n = 12345
print(len(str(n)))
```

39. Kadane Algorithm

```
arr = [-2,1,-3,4,-1,2,1,-5,4]
max_sum = curr = arr[0]
for i in arr[1:]:
    curr = max(i, curr+i)
```

```
    max_sum = max(max_sum, curr)
print(max_sum)
```

40. Two Pointer

```
arr = [1,2,3,4,6]
target = 6
l, r = 0, len(arr)-1
while l < r:
    s = arr[l]+arr[r]
    if s == target:
        print(arr[l], arr[r])
        break
    elif s < target:
        l += 1
    else:
        r -= 1
```

41. Sliding Window

```
arr = [1,2,3,4,5]
k = 3
window_sum = sum(arr[:k])
max_sum = window_sum
for i in range(k, len(arr)):
    window_sum += arr[i]-arr[i-k]
    max_sum = max(max_sum, window_sum)
print(max_sum)
```

42. Recursion Factorial

```
def fact(n):
    if n == 0:
        return 1
    return n * fact(n-1)
```

```
print(fact(5))
```

43. Recursion Fibonacci

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1)+fib(n-2)  
  
print(fib(6))
```

44. Lambda Map

```
arr = [1,2,3]  
print(list(map(lambda x:x*2, arr)))
```

45. Filter

```
arr = [1,2,3,4]  
print(list(filter(lambda x:x%2==0, arr)))
```

46. Zip

```
a = [1,2,3]  
b = ['a', 'b', 'c']  
print(list(zip(a,b)))
```

47. Enumerate

```
arr = ['x', 'y', 'z']  
for i,v in enumerate(arr):  
    print(i,v)
```

48. Exception Handling

```
try:  
    print(10/0)  
except Exception as e:  
    print(e)
```

49. Substring Check

```
s = "python interview"  
print("view" in s)
```

50. Generate Pairs

```
arr = [1,2,3]  
for i in range(len(arr)):  
    for j in range(i+1, len(arr)):  
        print(arr[i], arr[j])
```

51. Queue

```
queue = []  
queue.append(1)
```

```
queue.append(2)
print(queue.pop(0))
```

52. Stack

```
stack = []
stack.append(1)
stack.append(2)
print(stack.pop())
```

53. Reverse Words

```
s = "I love python"
print(" ".join(s.split()[::-1]))
```

54. First Non-Repeating Character

```
s = "aabbcddee"
for c in s:
    if s.count(c) == 1:
        print(c)
        break
```

55. Intersection Arrays

```
a = [1,2,3]
b = [2,3,4]
print([x for x in a if x in b])
```

56. Check Sorted

```
arr = [1,2,3,4]
print(arr == sorted(arr))
```

57. Prefix Sum

```
arr = [1,2,3,4]
prefix = []
s = 0
for i in arr:
    s += i
    prefix.append(s)
print(prefix)
```

58. Matrix Traversal

```
matrix = [[1,2],[3,4]]
for row in matrix:
    for val in row:
        print(val)
```

59. Generate Substrings

```
s = "abc"
for i in range(len(s)):
    for j in range(i+1, len(s)+1):
        print(s[i:j])
```

60. Word Frequency from Paragraph

```
import re

data = """He raced to the grocery store."""

words = re.findall(r'\b\w+\b', data.lower())

word_count = {}
for w in words:
    word_count[w] = word_count.get(w,0)+1

print(word_count)
```

Concept: Tokenization + frequency counting.

Pattern: Text Processing + HashMap

Time Complexity: $O(n)$

MOST IMPORTANT INTERVIEW PATTERNS

1. HashMap
2. Sliding Window
3. Two Pointer
4. Stack
5. Queue
6. Prefix Sum
7. Binary Search
8. Recursion
9. Sorting
10. Matrix Traversal
11. String Manipulation
12. Deduplication
13. Frequency Counting
14. Dynamic Programming
15. Flatten Nested Structures